Project Plan: Community Detection with QAOA

FYS5419 – Quantum Computing and Quantum Machine Learning
University of Oslo

Sindre Kampen Nesheim

May 7, 2026

Overview

This project investigates the use of the Quantum Approximate Optimization Algorithm (QAOA) [1] for community detection in graphs, with modularity maximization as the primary objective. The project spans both a quantum circuit implementation in Qiskit and a high-performance C++ implementation, enabling direct benchmarking between approaches. The work is designed with possible thesis reuse in mind, that is why I aim for a C++ implementation.

1 Problem Formulation

Community detection seeks to partition the nodes of a graph $G = (V, E)$ into groups (communities) such that intra-community edges are dense and inter-community edges are sparse. The standard quality measure is the *modularity* [4]:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \tag{1}$$

where A_{ij} is the adjacency matrix (symmetric over the diagonal), $k_i = \sum_j A_{ij}$ is the degree of node i, $m = \frac{1}{2} \sum_{ij} A_{ij}$ is the total number of edges, c_i is the community label of node i, and δ is the Kronecker delta.

1.1 QUBO Encoding

Modularity maximization is mapped to a Quadratic Unconstrained Binary Optimization (QUBO) problem [2]. For the two-community case, we assign each node a binary spin variable $x_i \in \{0, 1\}$ (equivalently $z_i = 2x_i - 1 \in \{-1, +1\}$), and the cost function becomes:

$$C(\mathbf{x}) = -\mathbf{x}^T Q_e \mathbf{x} + constrains, \tag{2}$$

where Q_e is the modularity matrix with entries $[Q_e]_{ij} = A_{ij} - k_i k_j/(2m)$. Constraints will be implemented as done in paper by Li et al. [3], with both a penalty if a node is in more than one cluster and if a cluster is without nodes.

1.2 Ising Hamiltonian

Substituting $x_i = (1 - Z_i)/2$ (with Z_i the Pauli-Z operator on qubit i), the modularity cost maps to the Ising cost Hamiltonian :

$$H_C \propto \frac{1}{4} \sum_{ij} Q_{ij} Z_i Z_j - \frac{1}{2} \sum_i \left(c_i + \sum_j Q_{ij} \right) Z_i, \tag{3}$$

where the linear terms arise from degree corrections. This is the operator whose expectation value the QAOA circuit is trained to minimize.

2 QAOA Implementation

2.1 Circuit Structure

The QAOA ansatz for p layers is:

$$|\psi_p\rangle = \prod_{k=1}^{p} e^{-i\beta_k H_M} \, e^{-i\gamma_k H_C} \, |+\rangle^{\otimes n}, \tag{4}$$

where the initial state $|+\rangle^{\otimes n}$ is prepared by applying Hadamard gates to all qubits in $|0\rangle^{\otimes n}$, and the two unitary layers are:

$$U_C(\gamma_i) = e^{-i\gamma_i H_C} = \prod_{ij} R_{z_i z_j} \frac{\gamma}{4} Q_{ij} \prod_i R_{z_i} \frac{\gamma}{2} \left(c_i + \sum_j Q_{ij} \right), \tag{5}$$

$$U_M(\beta_i) = e^{-i\beta_i H_M} = \prod_i R_x(2\beta_i), \tag{6}$$

with the mixer Hamiltonian $H_M = \sum_i X_i$.

2.2 Variational Optimization

The variational parameters $\boldsymbol{\theta} = (\boldsymbol{\gamma}, \boldsymbol{\beta})$ are optimized classically by minimizing the energy expectation value:

$$\langle H_C \rangle(\boldsymbol{\theta}) = \langle \psi_p(\boldsymbol{\theta}) | H_C | \psi_p(\boldsymbol{\theta}) \rangle. \tag{7}$$

The optimization will be carried out using scipy's optimization.

2.3 Circuit Depth and Quality Analysis

A central goal of the project is to study how the approximation ratio $r = \langle C \rangle / C_{\max}$ behaves as a function of circuit depth p and the variational parameters $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$. This includes:

- Sweeping $p = 1, 2, 3, 4$ and comparing convergence behavior.
- Visualizing the energy landscape $\langle H_C \rangle(\gamma, \beta)$ at fixed $p = 1$.
- Examining the recovered community partitions against ground-truth labels.

3 Implementation Strategy

3.1 Qiskit and IBM Quantum

The primary quantum simulation and execution will use Qiskit. Exact statevector simulation will be used for small graphs ($n \leq 16$) during development and benchmarking. For hardware experiments, circuits will be submitted to IBM's quantum computers via the Qiskit API. Circuit transpilation and noise-aware compilation will be handled through Qiskit's transpiler to match the target backend's native gate set and topology.

3.2 C++ Classical Implementation from Scratch

To complement the quantum approach and enable direct benchmarking, a classical C++ implementation will be written from scratch. This will include:

- Graph data structures (adjacency matrix and list representations).
- Construction and diagonalization of the modularity matrix Q_e.
- A shot based simulation.

The C++ code will be designed with HPC deployment in mind, as part of the broader ongoing research infrastructure for benchmarking optimization algorithms.

4 Benchmark Graphs

The project will be developed and tested on a progression of graphs of increasing size and complexity.

4.1 Starting Graphs

Networkx will be used to find and solve the initial small graphs. Examples of possible graphs is the Frutcht graphs (smallest cubical graphs), Padgett Florebtine families (real-world social networks) or stochastic block model (generative random graphs).

4.2 Target Graphs (Stretch Goals)

Time permitting, and subject to available quantum resources, the project aims to scale toward:

- The Zachary Karate Club graph (34 nodes).
- The IEEE 33-bus power network (33 nodes) as an applied engineering graph with known partition structure.

These graphs require either shot-based simulation or execution on actual IBM quantum hardware, and are therefore treated as stretch goals.

5 Expected Deliverables

1. A QAOA circuit implementation in Qiskit for modularity-based community detection.
2. A C++ codebase implementing graph construction, shot and state vector based QAOA.
3. A comparative analysis of QAOA approximation ratio vs. circuit depth p on small stochastic block model and real-world graphs.

4. A written report discussing the quality of recovered communities, the energy landscape geometry, and a discussion of scalability.

References

[1] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. *A Quantum Approximate Optimization Algorithm*. en. arXiv:1411.4028 [quant-ph]. Nov. 2014. DOI: 10.48550/arXiv.1411.4028. URL: http://arxiv.org/abs/1411.4028 (visited on 05/07/2026).

[2] Gary Kochenberger et al. "The unconstrained binary quadratic programming problem: a survey". en. In: *Journal of Combinatorial Optimization* 28.1 (July 2014), pp. 58–81. ISSN: 1382-6905, 1573-2886. DOI: 10.1007/s10878-014-9734-0. URL: http://link.springer.com/10.1007/s10878-014-9734-0 (visited on 05/07/2026).

[3] Wei Li et al. "Community detection by simulated bifurcation". en. In: ().

[4] M. E. J. Newman and M. Girvan. "Finding and evaluating community structure in networks". en. In: *Physical Review E* 69.2 (Feb. 2004), p. 026113. ISSN: 1539-3755, 1550-2376. DOI: 10.1103/PhysRevE.69.026113. URL: https://link.aps.org/doi/10.1103/PhysRevE.69.026113 (visited on 05/07/2026).